



School of Computer Science and Engineering

Winter Semester 2022-2023

Continuous Assessment Test – 2

SLOT: C2+TC2

Programme Name & Branch: B.Tech (CSE & Specializations)

Course Name & code: Data Structures and Algorithms & BCSE202L

Class Number (s): VL2022230505827/ 5830/5833/5835/5854/5863/5954/6219/6332/6588/6597/7613

Faculty Name (s):

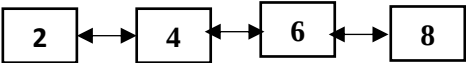
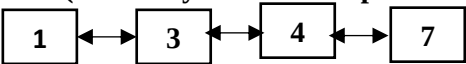
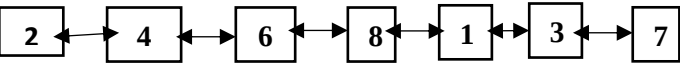
Exam Duration: 90 Min.

Answer ALL questions

Maximum Marks: 50

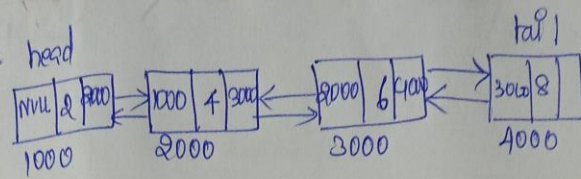
General instruction(s):

Wherever steps required, they must be clearly specified, if needed, with suitable diagrams.

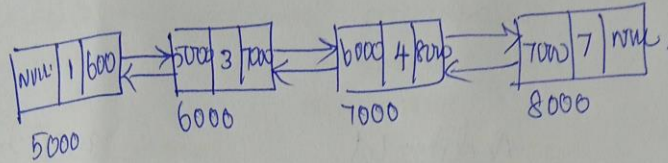
Q.No.	Question	Max Marks	CO
1.	How can a set (say, consisting integers as elements) be represented by a Doubly Linked list? Devise an algorithm (in pseudocode) to perform the operation UNION using the given input and illustrate its working in detail. Your pseudocode should be generic and work for any meaningful input. Following example is for understanding only. As repeated elements are not allowed in sets, you have to remove if any duplicated values present in the output set (which is represented by linked list). Your code should involve this operation (finding repeated elements and removing them) too. Input: Set 1 (in Doubly linked list representation) =  Set 2 (in Doubly linked list representation)=  Output: Set3 =  Answer Key:	10	CO2

① Union : head

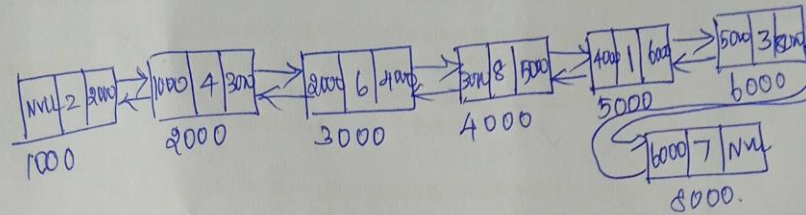
List1:



List2:



List3:



void main()

{

// List1;

insert(List1, 2);

insert(List1, 4);

insert(List1, 6);

insert(List1, 8);

// List2

insert(List2, 1);

insert(List2, 3);

insert(List2, 4);

insert(List2, 7);

union(List1, List2);

}

struct node

{ struct node *prev;

int data;

struct node *next;

};

struct node *List1=NULL, *List2=NULL;

struct node *head, *tail;

void insert(struct node *temp, int x)

{ struct node *p;

if (temp == NULL)

{

p = (struct node *) malloc(sizeof(struct node));

p->data = x;

p->prev = p->next = NULL;

head = p;

tail = p

} else

p = (struct node *) malloc(sizeof(struct node));

p->data = x;

tail->next = p->next;

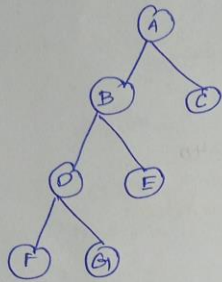
tail->next = p;

p->prev = tail;

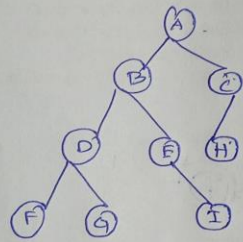
p = tail;

	<pre> void union (struct node *List1, struct node *List2) { struct node *temp = List1; while (temp != NULL) { insert (List3, temp->data); // Add all nodes of temp = temp->next; // first linked list } // to List3 while (List2 != NULL) { if (!search (List1, List2->data)) // Insert insert (List3, List2->data); // those nodes List2 = List2->next; // of List2 } // which is } // not present } // in List1 } </pre>		
2.	a) Draw a binary tree that is strictly binary tree but not a perfect binary tree (with min. height 3). b) Draw a binary tree that is not a strictly binary tree (with height at least 3). c) Draw an almost complete binary tree (also called as complete binary tree where in the binary tree, till the penultimate level, the tree is a perfect binary tree and at the last level, the nodes are filled from left to right) with height 4 which is not a strictly binary tree. Justify your claim (why is so and why is not so) for each of the above cases.	(6+4)	CO4

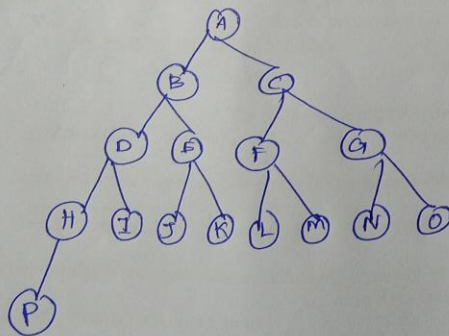
2) Strictly Binary Tree but not a Perfect Binary Tree.
height = 3.



2b) Binary Tree that is not a strictly Binary Tree.



2c) Almost complete Binary Tree not a strictly Binary Tree.
height = 4.



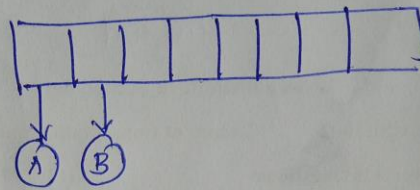
d) Draw the expression tree of the following postfix expression.

AB+C+DC+F+*GH+*

From the result of an expression tree, find the corresponding Prefix expression
(4 Marks)

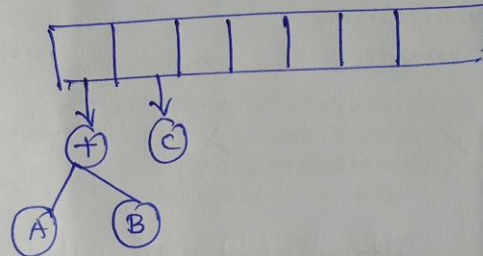
4) expression tree:

Stack:
input = operand then
push.

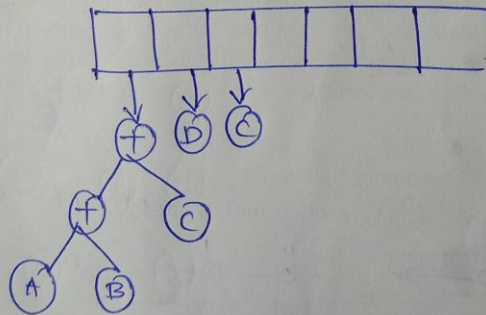


i/p = +

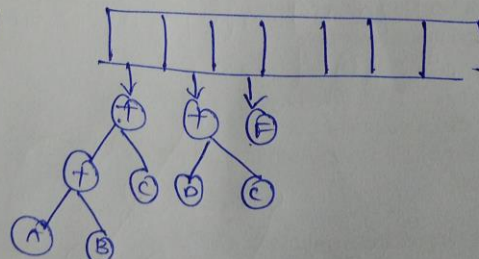
input = operator then
pop two sub tree.

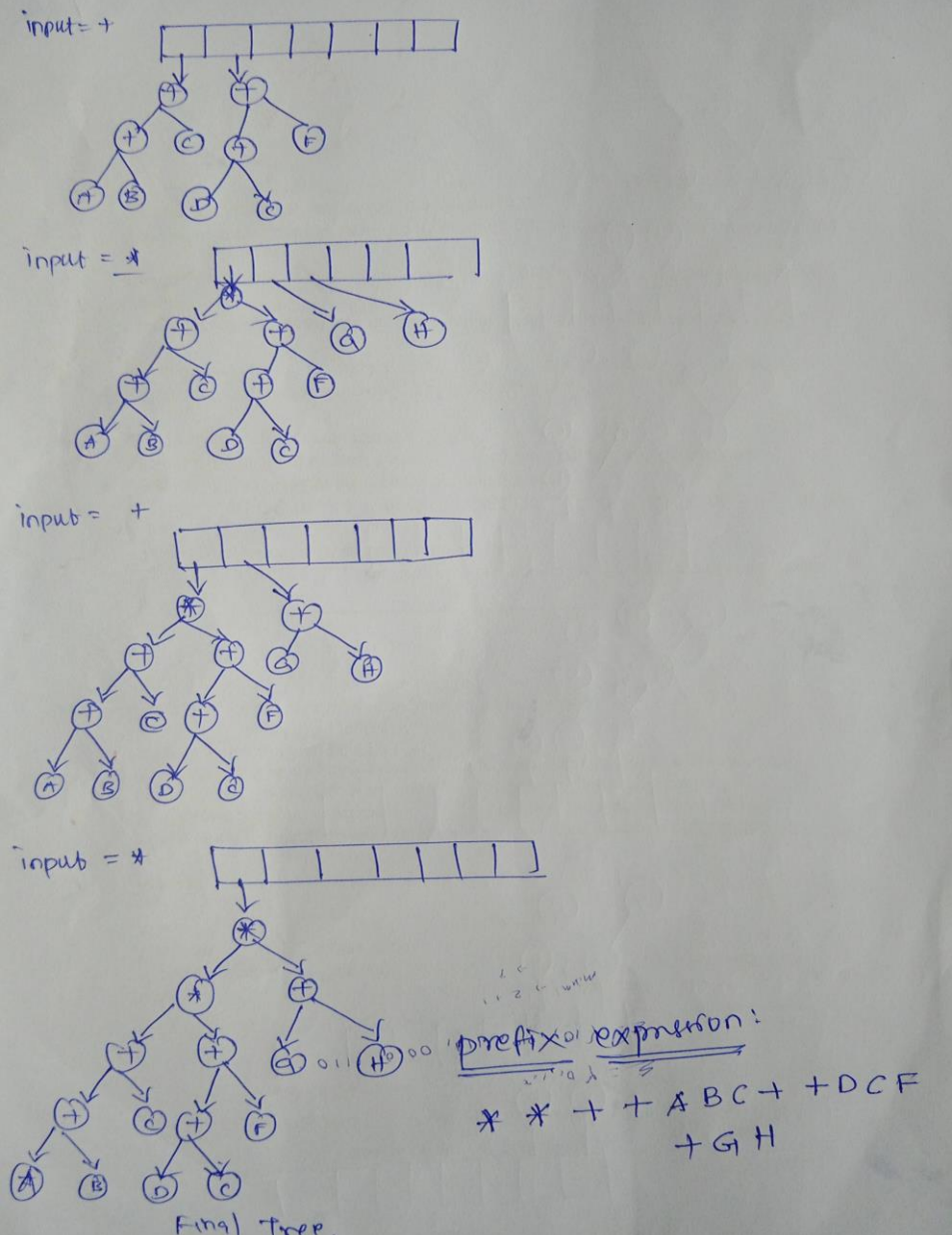


i/p = +



input = +



	 Final Tree. Prefix expression: $* * + + A B C + + D C F + G H$		
3.	a) Construct a tree for the given inorder and postorder traversals. Inorder : DGBAHEICF Postorder : GDBHIEFCA (4 Marks) b). Write an algorithm/pseudocode to count the number of terminal nodes and non-terminal (or internal) nodes in a resultant tree of Question 3a. (4 Marks)	10	CO4

3.

Ans:
(3a)

```

graph TD
    A((A)) --> B((B))
    A --> C((C))
    B --> D((D))
    D --> G((G))
    C --> E((E))
    C --> F((F))
    E --> H((H))
    E --> I((I))
  
```

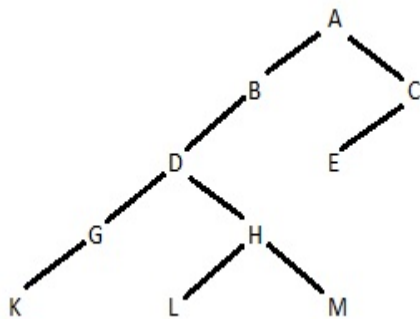
Binary Tree.

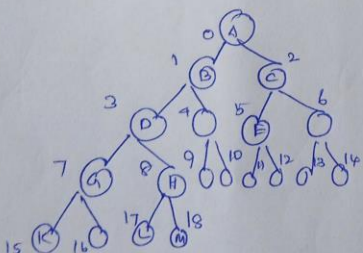
3(b)

```

int leafnodecount(struct node
                  *Tree)
{
    if (Tree == NULL)
        return 0;
    if (Tree->left == NULL &&
        Tree->right == NULL)
        return 1;
    return (countleafnode(Tree->
        left) +
        countleafnode(Tree->right));
}
  
```

c) Provide the array representation of the following binary tree.



	<pre>int countNonleaf(struct node *Tree) { if (Tree == NULL (Tree->left == NULL && Tree->right == NULL)) return 0; return 1 + countNonleaf(Tree->left) + countNonleaf(Tree->right); }</pre> 3 c)  Array representation. <table border="1" data-bbox="466 862 1305 967"><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td><td>H</td><td>I</td><td>J</td><td>K</td><td>L</td><td>M</td><td>N</td><td>O</td><td>P</td><td>Q</td><td>R</td><td>S</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td><td>16</td><td>17</td><td>18</td></tr></table>	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18		
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S																							
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18																							
	(2 Marks)																																								
4.	Write an (efficient) algorithm/pseudocode to merge the following two sorted array into a single sorted (in descending order). Apply your code to the below two sorted arrays. What is the time-complexity of your algorithm? Input A = [10,8,5,2], B = [13,11,9,7,6,3] Output C = [13,11,10,9,8,7,6,5,3,2]	10	CO3																																						

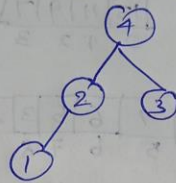
	4) <u>Merge sort:</u> A <table border="1" style="border-collapse: collapse; margin: 0 auto;"><tr><td>10</td><td>8</td><td>5</td><td>2</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td></tr></table> B <table border="1" style="border-collapse: collapse; margin: 0 auto;"><tr><td>13</td><td>11</td><td>9</td><td>7</td><td>6</td><td>3</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table> C <table border="1" style="border-collapse: collapse; margin: 0 auto;"><tr><td>13</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>3</td><td>2</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td></tr></table> <pre>void mergearray(int A[], int B[], int s1, int s2) { int s3 = s1 + s2; int C[s3], i = 0, j = 0, k = 0; while (i < s1 && j < s2) { if (A[i++] > B[j++]) C[k++] = A[i-1]; else C[k++] = B[j-1]; } if (i >= s1) while (j < s2) C[k++] = B[j++]; if (j >= s2) while (i < s1) C[k++] = A[i++]; } //end merge sort.</pre>	10	8	5	2	0	1	2	3	13	11	9	7	6	3	0	1	2	3	4	5	13	11	10	9	8	7	6	5	3	2	0	1	2	3	4	5	6	7	8	9		
10	8	5	2																																								
0	1	2	3																																								
13	11	9	7	6	3																																						
0	1	2	3	4	5																																						
13	11	10	9	8	7	6	5	3	2																																		
0	1	2	3	4	5	6	7	8	9																																		
5.	a) Consider the array A = [4,1,3,2,16,9,10,14,8,7] Build the MAX-HEAP for the input array A, (5 Marks) b) Perform the following operations using the Heap data structure that is created in Question 5a) and draw the resultant Heap tree after each of the Operations i) Delete 16 ii) Insert 22 iii) Delete 22 (5 Marks)	10	CO5																																								

(5) Build Max Heap: $A = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$

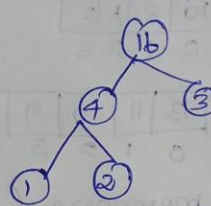
insert (4, 1, 3)



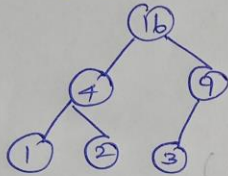
insert(2)



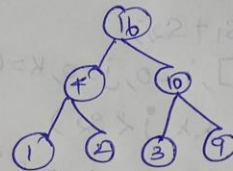
insert(16)



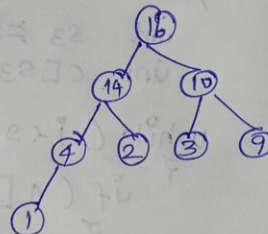
insert(9)



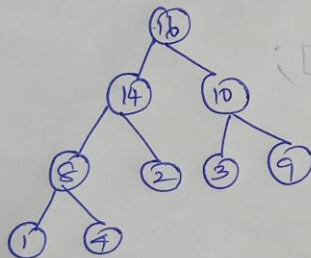
insert(10)



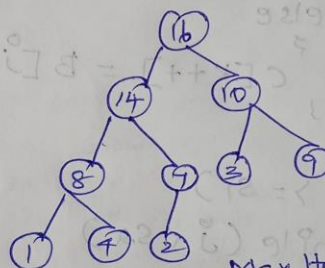
insert(14)



insert(8)

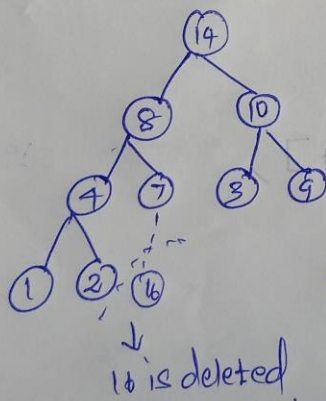


insert(7)

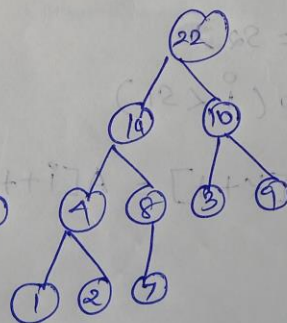


Max Heap

(i) Delete(16)



(ii) Insert(22)



(iii) Delete(22)

